CSE 219 — Signals and Linear Systems

Lec 3–4 · Fast Fourier Transform: Radix-2, Matrix FFT, Fast Convolution, Bluestein

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Ashraf Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — FFT questions and their weight
Lesson 5 — Radix-2 Decimation-in-Time FFT — derivation, butterflies, bit reversal, complexity, a full 8-point example
Lesson 6 — Radix-2 Decimation-in-Frequency FFT — first/second-half split, DIF butterfly, bit-reversed outputs
Lesson 7 — The Matrix (Cooley–Tukey / Bailey) Formulation — column-major input, row DFTs, twiddles, column DFTs
Lesson 8 — Fast Convolution and Bluestein's FFT for Prime N — O(N log N) filtering; chirp-z identity; $M \geq 2N - 1$
Quick Notes — FFT — merged revision notes + key diagrams

> **NOTE** Slide numbers are PDF page numbers of Lecture 3 - Fast Fourier Transform.pdf (19 pages, “Lec 3”) and Lecture 4 - Advanced FFT.pdf (24 pages, “Lec 4”); the printed slide number is one less.

Exam Map — What the TF Papers Ask

Session (paper)	Questions answered by this note
2023-24 (CSE 219)	Q5(a) 8-point DFT of {1,2,3,4,3,2} by radix-2 DIT FFT with all butterflies (15) · Q7(a) 2×4 Cooley–Tukey (matrix) DFT of {1,2,3,4,9,0,7,8}, steps i–iii (15) · Q7(b) Chirp-z (Bluestein) derivation; why padding M ≥ 2N−1 (10)
2022-23 (CSE 219)	Q4(c) Compare computational complexity of direct DFT and radix-2 FFT (15)
2017-18 (CSE 311)	Q3(b) DIF FFT formulation; 8-point signal-flow graph; bit-reversed outputs (24)
2014-15 (CSE 311)	Q7(c) Recurrence relationship of the FFT (15)

EXAM FOCUS CSE 219 2023-24 spent 40 marks here: an 8-point radix-2 DIT FFT with every butterfly (Q5(a), 15), a 2 × 4 matrix FFT (Q7(a), 15) and the Bluestein derivation (Q7(b), 10); 2022-23 asked the DFT vs FFT complexity (Q4(c), 15). Practise the 8-point DIT until you can do it in 12 minutes, and know W_8 by heart.

Lesson 5 — Radix-2 Decimation-in-Time FFT

Lec 3 sl. 4–15, 19

1. Why: the Naive DFT Is O(N²) (sl. 4)

Each of the N outputs needs N complex multiplications and N − 1 additions: N² multiplications in all. One second of audio (N = 44 100) needs ≈ 2 × 10⁹ operations.

2. The Even/Odd Split (sl. 7–9)

```
X[k] = Σ_{r} x[2r]W_N^{2rk} + Σ_{r} x[2r+1]W_N^{(2r+1)k},   W_N^{2rk} = W_{N/2}^{rk}
     = G[k] + W_N^k H[k]     G = N/2-point DFT of the evens,  H = N/2-point DFT of the odds
G[k + N/2] = G[k], H[k + N/2] = H[k] (period N/2), and W_N^{k+N/2} = -W_N^k, so for 0 ≤ k <
N/2:
```

X[k] = G[k] + W_N^k H[k] X[k + N/2] = G[k] − W_N^k H[k]

Recurrence (the “recurrence relationship of the FFT”): an N-point DFT = two N/2-point DFTs + N/2 butterflies. Repeat until 2-point DFTs (X[0] = x[0] + x[1], X[1] = x[0] − x[1]).

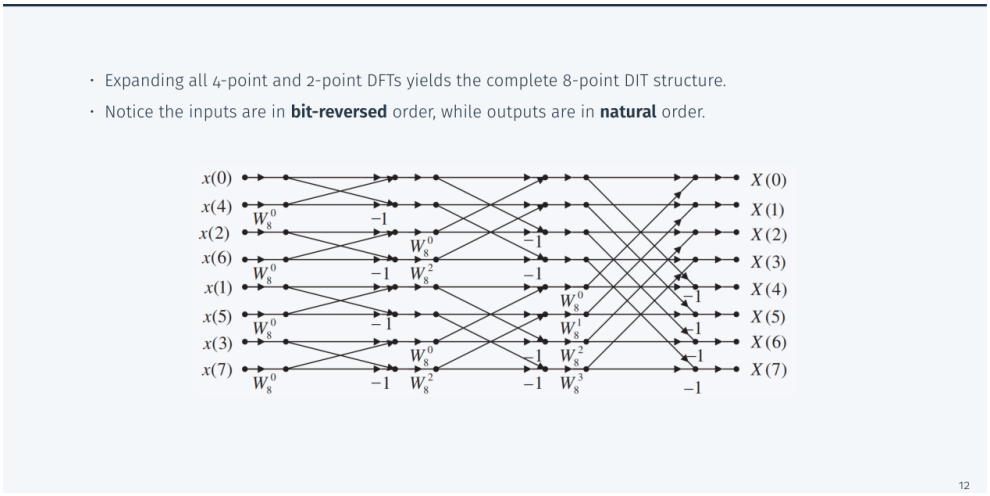


Figure 5.1 — Full 8-point DIT flow graph: bit-reversed inputs, natural-order outputs (Lec 3, sl. 13)

3. Bit-Reversed Input Order (sl. 14)

```
n:      0    1    2    3    4    5    6    7
binary 000  001  010  011  100  101  110  111  → reversed 000 100 010 110 001 101 011 111
order:  x[0] x[4] x[2] x[6] x[1] x[5] x[3] x[7]   (evens before odds, then even-evens before
odd-evens, …)
```

4. Complexity (sl. 15)

	Direct DFT	Radix-2 FFT
complex multiplications	N^2	$(N/2)\log_2 N$ (one per butterfly)
complex additions	$N(N-1)$	$N \log_2 N$
stages / butterflies per stage	—	$\log_2 N$ stages, N/2 butterflies each
N = 8	64	12 (5.3× fewer)
N = 1024	1 048 576	5 120 (≈ 205×)
N = 2^{20}	1.1×10^{12}	1.05×10^{7} (≈ 10^5×)

Plus O(N) for the bit-reversal permutation. Requires N = 2^m; otherwise zero-pad to the next power of 2 (fine for spectra — it only interpolates the DTFT — but it changes the period of a periodic signal) (sl. 19).

MEMORIZE $W_8^0 = 1$, $W_8^1 = (1 - j)/\sqrt{2} = 0.7071 - 0.7071j$, $W_8^2 = -j$, $W_8^3 = -(1 + j)/\sqrt{2} = -0.7071 - 0.7071j$; $W_4^1 = -j$. A butterfly: top = g + W·h, bottom = g − W·h.

Exam Approach

1. Zero-pad to a power of 2; write the bit-reversed input order.
2. Stage 1: 2-point butterflies (W = 1) on adjacent pairs.
3. Stage 2: 4-point combines with $W_4^0 = 1$, $W_4^1 = -j$ (pairs 2 apart).
4. Stage 3: 8-point combines with $W_8^0 \ldots W_8^3$ (pairs 4 apart). Show every intermediate value.
5. Check: X[0] = Σx, X[4] = Σ(−1)ⁿx[n], X[8 − k] = X*[k] for real x.

Practice — Lesson 5

(a) Actual — CSE 219 2023-24 Q5(a). (b) Actual — CSE 219 2022-23 Q4(c). (c) Actual — CSE 311 2014-15 Q7(c). (d) Practice set 1.2.

(a) x[n] = {1, 2, 3, 4, 3, 2}. Compute the 8-point DFT with the radix-2 DIT FFT, showing all intermediate stages with a proper butterfly diagram. (15)

(b) Compare the computational complexities of the DFT and the radix-2 FFT. (15)

(c) Derive the recurrence relationship of the FFT. (15)

(d) 4-point DIT FFT of {2, 1, −3, 4}.

✓ Solution

```
(a) Pad: x = {1, 2, 3, 4, 3, 2, 0, 0}.  Bit-reversed order: x0 x4 x2 x6 x1 x5 x3 x7 = 1, 3, 3,
0, 2, 2, 4, 0

Stage 1 (2-point, W = 1):
  (x0,x4) = (1,3)  → 1+3 = 4,   1−3 = −2
  (x2,x6) = (3,0)  → 3,  3
  (x1,x5) = (2,2)  → 4,  0
  (x3,x7) = (4,0)  → 4,  4
Stage 2 (4-point, W₄⁰ = 1, W₄¹ = −j):
  evens  G: (4, −2 | 3, 3):  G0 = 4+3 = 7,  G1 = −2 + (−j)3 = −2−3j,  G2 = 4−3 = 1,  G3 = −2 −
(−j)3 = −2+3j
  odds   H: (4, 0 | 4, 4):  H0 = 4+4 = 8,  H1 = 0 + (−j)4 = −4j,     H2 = 4−4 = 0,  H3 = 0 −
(−j)4 = 4j
Stage 3 (8-point, X[k] = G[k] + W₈^k H[k], X[k+4] = G[k] − W₈^k H[k]):
  k=0: W·H = 8                          X[0] = 7+8 = 15            X[4] = 7−8 = −1
  k=1: W₈¹·(−4j) = (0.707−0.707j)(−4j) = −2.828−2.828j
                                        X[1] = −4.828−5.828j       X[5] = 0.828−0.172j
  k=2: W₈²·0 = 0                        X[2] = 1                   X[6] = 1
  k=3: W₈³·(4j) = (−0.707−0.707j)(4j) = 2.828−2.828j
                                        X[3] = 0.828+0.172j        X[7] = −4.828+5.828j

X = {15, −4.828−5.828j, 1, 0.828+0.172j, −1, 0.828−0.172j, 1, −4.828+5.828j}
Checks: X[0] = Σx = 15 ✓;  X[4] = 1−2+3−4+3−2 = −1 ✓;  X[7] = X*[1], X[5] = X*[3] ✓ (numpy
agrees)
```

(b) Direct DFT: N² complex multiplications and N(N − 1) complex additions (each of N outputs sums N products). Radix-2 FFT: $\log_2 N$ stages × N/2 butterflies, each with one complex multiplication and two additions ⇒ $(N/2)\log_2 N$ multiplications and $N \log_2 N$ additions (+ O(N) bit reversal); trivial twiddles (±1, ±j) reduce it further. Speed-up $N^2/((N/2)\log_2 N) = 2N/\log_2 N$: 5.3 for N = 8, ≈ 205 for N = 1024, ≈ 10^5 for N = 2^{20} — a day of computation becomes seconds. Limitation: N must be a power of 2 (zero-pad, or use mixed-radix/Bluestein).

(c) The split in §2: X_N[k] = $G_{N/2}$[k] + W_N^k $H_{N/2}$[k], X_N[k + N/2] = $G_{N/2}$[k] − W_N^k $H_{N/2}$[k], applied recursively to G and H. Cost recurrence: C(N) = 2C(N/2) + N/2 multiplications, C(2) = 0 (W = 1) ⇒ C(N) = (N/2)$\log_2$N.

```
(d) order x0 x2 x1 x3 = 2, −3, 1, 4;  stage 1: (2−3, 2+3) = (−1, 5);  (1+4, 1−4) = (5, −3)
    stage 2: X0 = −1 + 5 = 4;  X1 = 5 + (−j)(−3) = 5+3j;  X2 = −1 − 5 = −6;  X3 = 5 − (−j)(−3)
= 5−3j
```

Lesson 6 — Radix-2 Decimation-in-Frequency FFT

Lec 3 sl. 16–18

```
Split the time sum into first and second halves:
X[k] = Σ_{n=0}^{N/2-1} [x[n] + (-1)^k x[n + N/2]] W_N^{kn}          (W_N^{kN/2} = (-1)^k)
even k = 2m:  X[2m]   = Σ (x[n] + x[n+N/2]) W_{N/2}^{mn}           = DFT_{N/2}{a(n)}
odd  k = 2m+1: X[2m+1] = Σ [(x[n] - x[n+N/2])W_N^n] W_{N/2}^{mn}   = DFT_{N/2}{b(n)}
a(n) = x[n] + x[n + N/2],   b(n) = (x[n] - x[n + N/2])·W_N^n      (add/subtract first, THEN
twiddle)
```

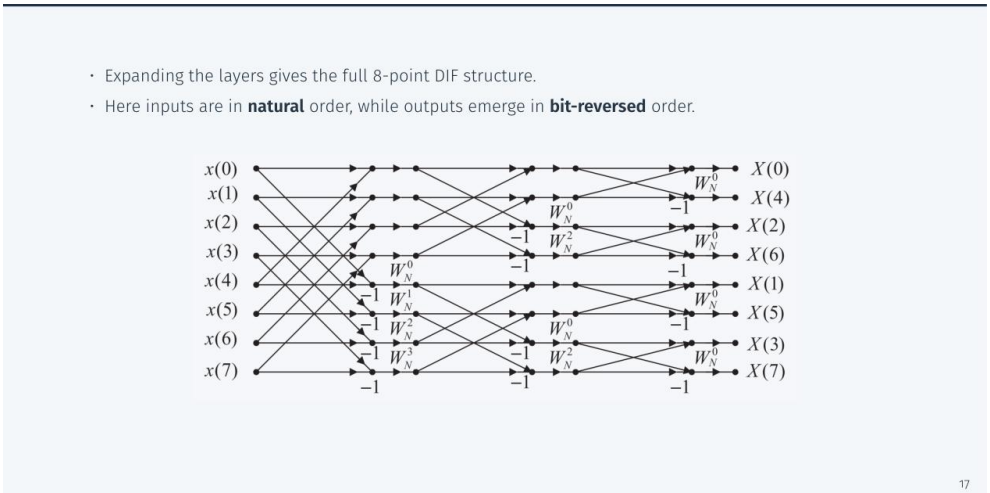


Figure 6.1 — 8-point DIF flow graph: natural-order inputs, bit-reversed outputs (Lec 3, sl. 18)

	DIT	DIF
split	time index even/odd	time first half/second half (= output even/odd)
butterfly	multiply by W, then add/subtract	add/subtract, then multiply by W
input order	bit-reversed	natural
output order	natural	bit-reversed
cost	$(N/2)\log_2 N$ mult.	$(N/2)\log_2 N$ mult.

Practice — Lesson 6

(a) Actual — CSE 311 2017-18 Q3(b). (b) Constructed.

(a) Formulate the decimation-in-frequency FFT; draw the 8-point signal-flow graph and explain why the outputs come out in bit-reversed order. (24)

(b) Run the first DIF stage on x = {1, 2, 3, 4, 3, 2, 0, 0}.

✓ Solution

(a) Derivation as in the grey box; apply it again to a(n) and b(n) (N/4-point blocks), then to 2-point butterflies. The 8-point graph (Figure 6.1): stage 1 pairs (n, n + 4) with twiddles $W_8^0 \ldots W_8^3$ on the difference branch; stage 2 pairs (n, n + 2) inside each half with W_4^0, W_4^1; stage 3 pairs (n, n + 1) with W = 1. Output order: stage 1 separates even k (top) from odd k (bottom), stage 2 separates k mod 4, …, so the final position of X[k] is k with its bits reversed: X[0], X[4], X[2], X[6], X[1], X[5], X[3], X[7].

```
(b) a(n) = x[n] + x[n+4] = {1+3, 2+2, 3+0, 4+0} = {4, 4, 3, 4}  → its 4-point DFT gives X[0],
X[2], X[4], X[6]
    b(n) = (x[n] - x[n+4])W_8^n = {-2·1, 0·W_8, 3·(-j), 4·W_8^3} = {-2, 0, -3j, -2.828-2.828j}  →
X[1], X[3], X[5], X[7]
    check: DFT_4{4,4,3,4} = {15, 1, -1, 1} = X[0], X[2], X[4], X[6] ✓ (Lesson 5)
```

Lesson 7 — The Matrix (Cooley–Tukey / Bailey) Formulation

Lec 4 sl. 5–14

Radix-2 as a table (sl. 6–8): put x into a 2 × N/2 matrix **column-major** ($n = 2n_2 + n_1$: row 0 = evens, row 1 = odds); DFT each row (→ G, H); multiply row n_1 by $W_N^{n_1 k_2}$; 2-point DFT down each column (the butterflies); read the result **row-major** ($k = (N/2)k_1 + k_2$).

```
General N = N₁N₂ (sl. 11–13):  n = N₁n₂ + n₁ (column-major),  k = N₂k₁ + k₂ (row-major)
(N₁n₂ + n₁)(N₂k₁ + k₂) = Nn₂k₁ + N₁n₂k₂ + N₂n₁k₁ + n₁k₂;  W_N^{Nn₂k₁} = 1
X[N₂k₁ + k₂] = Σ_{n₁} W_N^{n₁k₂} [Σ_{n₂} x[N₁n₂ + n₁]W_{N₂}^{n₂k₂}] W_{N₁}^{n₁k₁}
               twiddle   └ N₂-point DFT of row n₁ ┘   N₁-point DFT down columns
```

Bailey's four-step FFT (sl. 14): (1) arrange $N_1 \times N_2$ column-major; (2) N_2-point DFT of each row; (3) multiply entry (n_1, k_2) by $W_N^{n_1 k_2}$; (4) N_1-point DFT of each column; read row-major. Recursing on the factors gives O(N log N) for any N with small prime factors (radix-3: $N_1 = 3$, sl. 9–10).

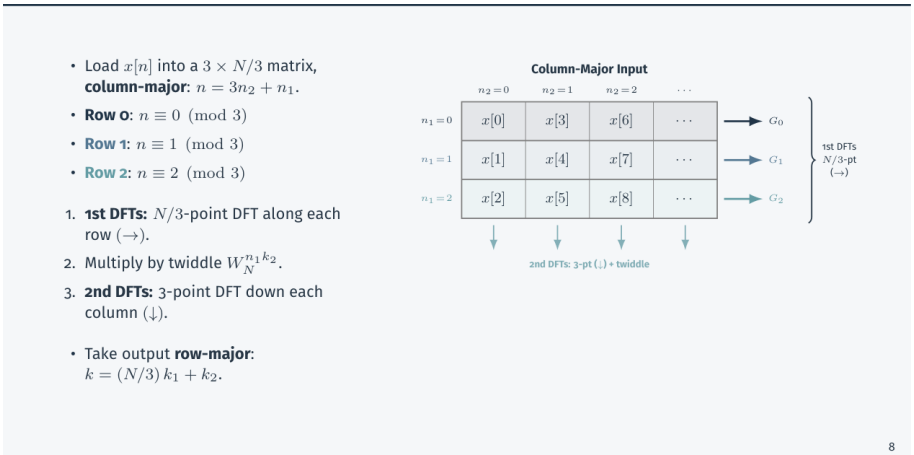


Figure 7.1 — Radix-3 as a 3 × N/3 table (Lec 4, sl. 10)

Practice — Lesson 7

(a) Actual — CSE 219 2023-24 Q7(a). (b) Practice set 2.2.

(a) Using the radix-2 Cooley–Tukey 2D matrix formulation, compute the DFT of x = {1, 2, 3, 4, 9, 0, 7, 8}: (i) arrange in the 2 × 4 column-major matrix (evens/odds) and compute the 4-point DFT of each row (matrix G); (ii) twiddles $W_8^{n_1 k_2}$ in a + jb form; H = G ∘ W; (iii) 2-point DFTs down each column, read row-major. (15)

(b) Same method for x = {1, 3, −1, 2, 0, 4, 2, 1}.

✓ Solution

```
(i)  Row 0 (n₁ = 0, evens): x0 x2 x4 x6 = 1, 3, 9, 7      Row 1 (odds): x1 x3 x5 x7 = 2, 4, 0,
8
     4-point DFT (W₄ = −j): [a,b,c,d] → [a+b+c+d, (a−c) − j(b−d), a−b+c−d, (a−c) + j(b−d)]
     G row 0: [20, −8+4j, 0, −8−4j]          G row 1: [14, 2+4j, −10, 2−4j]
(ii) Twiddles W₈^{n₁k₂}: row 0: 1, 1, 1, 1;   row 1: 1, 0.7071−0.7071j, −j, −0.7071−0.7071j
     H row 0: [20, −8+4j, 0, −8−4j]
     H row 1: [14, (2+4j)(0.7071−0.7071j) = 4.2426+1.4142j, (−10)(−j) = 10j,
(2−4j)(−0.7071−0.7071j) = −4.2426+1.4142j]
(iii) Column 2-point DFTs (top + bottom, top − bottom):
     k₂=0: 20 ± 14            → 34, 6
     k₂=1: (−8+4j) ± (4.2426+1.4142j) → −3.7574+5.4142j,  −12.2426+2.5858j
     k₂=2: 0 ± 10j            → 10j, −10j
     k₂=3: (−8−4j) ± (−4.2426+1.4142j) → −12.2426−2.5858j,  −3.7574−5.4142j
     Row-major (k = 4k₁ + k₂): row k₁ = 0 gives X[0..3], row k₁ = 1 gives X[4..7]

     k    Re X[k]     Im X[k]
     0    34           0
     1    −3.7574      5.4142
     2     0          10
     3   −12.2426     −2.5858
     4     6           0
     5   −12.2426      2.5858
     6     0         −10
     7    −3.7574     −5.4142        (= numpy.fft.fft ✓; X[8−k] = X*[k] ✓)
```

(b) Rows {1, −1, 0, 2} and {3, 2, 4, 1}; the same four steps reproduce numpy's 8-point DFT (checked in verify_ashraf12.py).

Lesson 8 — Fast Convolution and Bluestein's FFT for Prime N

Lec 4 sl. 16–22

1. Fast Convolution (sl. 16–17)

```
1) N ≥ L + M − 1, next power of 2   2) zero-pad x, h   3) X = FFT(x), H = FFT(h)   4) Y = XH
5) y = IFFT(Y)
Cost: 3 FFTs + N products = O(N log N) vs O(LM) direct.  1 s of audio with a 1 s reverb (L = M
= 44 100):
direct ≈ 1.94 × 10⁹;  fast: N = 2¹⁷ = 131 072, 3N log₂N ≈ 6.7 × 10⁶  ⇒ ≈ 300× faster
```

2. Bluestein's Algorithm (sl. 19–22)

Cooley–Tukey needs a composite N; for a large prime N it falls back to O(N²). Bluestein turns the DFT into a linear convolution, which we can compute fast for any length.

```
Chirp-z identity:  (k − n)² = k² − 2kn + n²  ⇒  kn = [n² + k² − (k − n)²]/2
                   W_N^{kn} = W_{2N}^{2kn} = W_{2N}^{k²} W_{2N}^{n²} W_{2N}^{-(k-n)²}
X[k] = Σ_n x[n]W_N^{kn} = W_{2N}^{k²} Σ_{n=0}^{N-1} [x[n]W_{2N}^{n²}]·W_{2N}^{-(k-n)²}
     = W_{2N}^{k²} (a * b)[k],   a[n] = x[n]W_{2N}^{n²} (0 ≤ n ≤ N-1),  b[n] = W_{2N}^{-n²}
(-(N-1) ≤ n ≤ N-1)
```

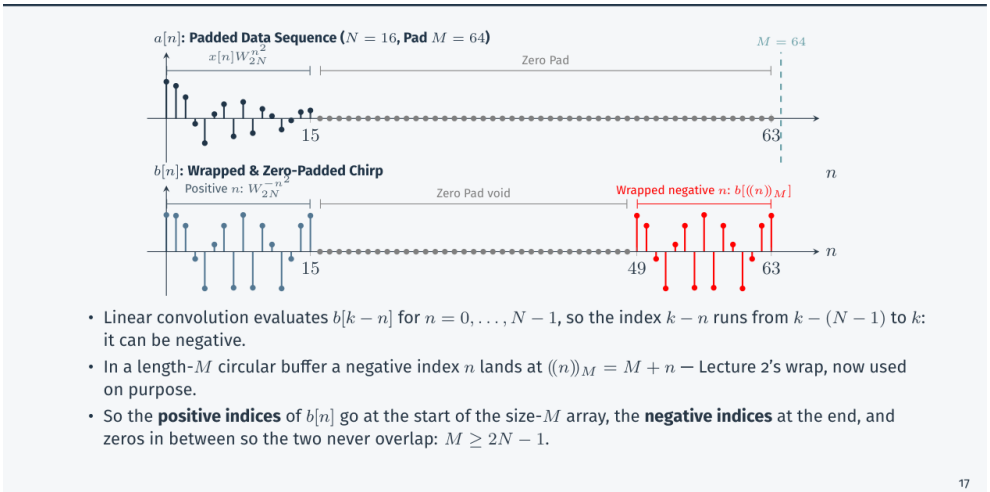


Figure 8.1 — Bluestein padding: positive chirp indices at the start, negative ones wrapped to the end (Lec 4, sl. 21)

```
Padding: k − n runs from −(N−1) to N−1, so b has 2N − 1 samples. In a circular buffer of length
M an index −m
lands at M − m. Put b[0..N−1] at the start, b[−(N−1)..−1] at the end, zeros between. They must
not overlap:
M ≥ 2N − 1 (choose M = 2^{⌈log₂(2N−1)⌉} so radix-2 FFTs apply — the prime N never touches the
transform length).
Steps: A = FFT_M(a padded), B = FFT_M(b wrapped), y = IFFT_M(AB), X[k] = W_{2N}^{k²}y[k], k =
0…N−1.
```

Practice — Lesson 8

(a) Actual — CSE 219 2023-24 Q7(b). (b) Practice set 3.2.

(a) Derive the N-point DFT as a linear convolution using the chirp-z identity kn = n²/2 + k²/2 − (k − n)²/2, outline its fast-convolution implementation and explain the padding requirement M ≥ 2N − 1. (10)

(b) Use Bluestein's construction to compute the 3-point DFT of {2, −1, 3}.

✓ Solution

(a) Derivation and padding argument in the two grey boxes above. Implementation: form a[n] (N samples) and the chirp b[n] (2N − 1 samples); pick M ≥ 2N − 1, a power of 2; zero-pad a to M; place b circularly (wrapped negative indices); radix-2 FFT both; multiply; IFFT; multiply the first N outputs by the post-chirp $W_{2N}^{k^2}$. Cost O(M log M) = O(N log N). Why M ≥ 2N − 1: a circular convolution of length M equals the needed linear values only if the wrapped negative-index part of b and its positive part do not overlap (and the wrap does not alias outputs k = 0…N − 1); with M < 2N − 1 some chirp samples land on top of others.

```
(b) N = 3, W₆ = e^{−jπ/3}: a[n] = x[n]W₆^{n²} = {2, −1·W₆¹, 3·W₆⁴} = {2, −0.5+0.866j,
−1.5+2.598j}
    b[n] = W₆^{−n²}, n = −2…2;  M = 8;  result X = {4, 1+3.464j, 1−3.464j} = direct DFT ✓
```

Quick Notes — FFT

Revision points for Lessons 5–8.

L5 · DIT

- X[k] = G + W^kH, X[k+N/2] = G − W^kH; bit-reversed input.
- Cost: N^2 vs $(N/2)\log_2 N$ multiplications; N = 8: 64 vs 12.
- {1,2,3,4,3,2,0,0} → {15, −4.828−5.828j, 1, 0.828+0.172j, −1, …}.

L6 · DIF

- a = x[n] + x[n+N/2] → even k; b = (x[n] − x[n+N/2])W^n → odd k; output bit-reversed.

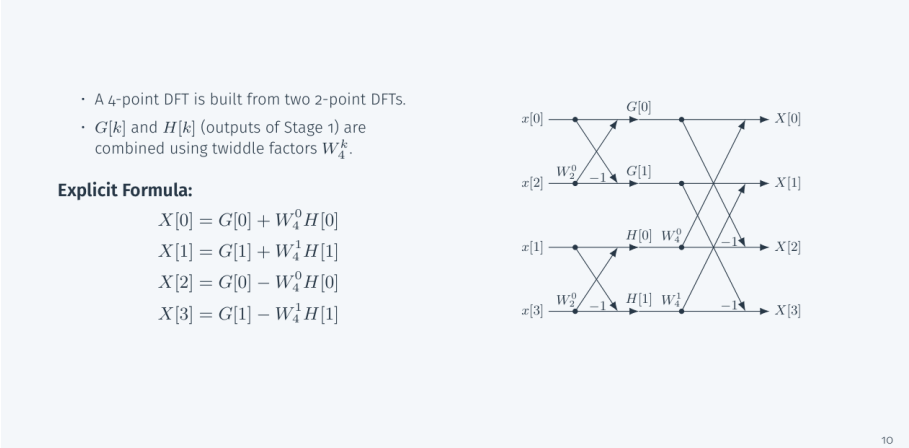
L7 · Matrix FFT

- n = $N_1 n_2 + n_1$ column-major; rows: N_2-pt DFT; twiddle $W_N^{n_1 k_2}$; columns: N_1-pt DFT; read $k = N_2 k_1 + k_2$.

L8 · Fast conv / Bluestein

- Pad to N ≥ L + M − 1 (power of 2), FFT, multiply, IFFT.
- $kn = [n^2 + k^2 - (k-n)^2]/2 \Rightarrow X = W_{2N}^{k^2}(a * b)$; M ≥ 2N − 1.

Key Diagrams to Reproduce



4-point DIT butterfly structure (Lec 3, sl. 11)